Import Statements

In [2]:
```
import os
import custom_cmap
from PIL import Image
from IPython.display import display

import matplotlib.pyplot as plt
import matplotlib as mpl
import pandas as pd

from moria import reduce
from pathlib import Path

from astropy.visualization import PercentileInterval
from astropy.io import fits
from astropy.visualization import LogStretch, ImageNormalize
import plotly.express as px
import numpy as np
from astropy.visualization import ImageNormalize, AsinhStretch, SqrtStretch, LogStretch, PowerStretch

plt.rcParams.update({'font.size':25})
plt.rc('text', usetex=True)
%config InlineBackend.figure_format = 'retina'
%load_ext autoreload
%autoreload 2
```

```
The autoreload extension is already loaded. To reload it, use:
  %reload_ext autoreload
```

Understanding the main directory.

The data directory should be organized as follows. You can look at the sample data folder to understand the directory strucutre you need.

00.DATA

01.XYM
02.CMD
03.LOC_TRANS
04.PSF_EXTRACT
05.COORD_TRANS(OPTIONAL)
06.FIT
07.CALIBRATION
All the necessary scripts are included in the sample data directory. The simplest way to run MORIA on your target is to copy the entirety of the "data" folder to wherever you wish to conduct your analysis. Start by putting your exposures in "data/00.DATA", then beginning with the demo.

After you finish running this notebook, you will be able to use the "_flc" exposures to generate an output image stack in the F814W and F606W HST filters.

NOTE: chmod +x program.src is a useful command to use whenever permissions are denied for a script.

In [3]:
```
#This is the directory where you are processing your data. This does not point at MORIA.
directory = os.getcwd()
```

Data Preparation

MORIA runs the "Makefile" in fortran_compile directly upon pip installation.

If you have different fortran dependincies, you will have to edit the Makefile.txt in fortran_compile, and then uncomment the code_directory to point to src/fortran_compile. Once you finish compiling the files above, run this cell below to put the fortran scripts in their appropriate directories:

In [3]:
```
reduce.data_prep_early(directory)
```

STEP 1

Download the _flc files. Place them appropriately in the F814W and F606W directories in 00.DATA.

The cell below will convert the _flc files to _WJ2 files using run_convert_C1K1C.src
These "_flc" exposures include the charge-transfer efficiency (CTE) corrections. MORIA first converts these "_flc" exposures into "_WJ2" files to convert to a full-chip co-ordiante system that are placed into appropriate reference geometry for subsequent steps.

In [4]:
```
reduce.run_xgf_conversion(directory)
```

```
NARG: FILENAME:            1  iffp02bxq_WJ2.fits
NARG: FILENAME:            2  iffp02byq_WJ2.fits
NARG: FILENAME:            3  iffp02c0q_WJ2.fits
NARG: FILENAME:            4  iffp02c1q_WJ2.fits
NARG: FILENAME:            5  iffp02c2q_WJ2.fits
NARG: FILENAME:            6  iffp02c3q_WJ2.fits
NARG: FILENAME:            7  iffp02c4q_WJ2.fits
NARG: FILENAME:            8  iffp02c5q_WJ2.fits
NARG: FILENAME:            9  iffp02c6q_WJ2.fits
NARG: FILENAME:           10  iffp02c7q_WJ2.fits
NARG: FILENAME:            1  iffp02bhq_WJ2.fits
NARG: FILENAME:            2  iffp02biq_WJ2.fits
NARG: FILENAME:            3  iffp02bjq_WJ2.fits
NARG: FILENAME:            4  iffp02bkq_WJ2.fits
NARG: FILENAME:            5  iffp02blq_WJ2.fits
NARG: FILENAME:            6  iffp02bmq_WJ2.fits
NARG: FILENAME:            7  iffp02bnq_WJ2.fits
NARG: FILENAME:            8  iffp02boq_WJ2.fits
NARG: FILENAME:            9  iffp02bpq_WJ2.fits
NARG: FILENAME:           10  iffp02bqq_WJ2.fits
NARG: FILENAME:           11  iffp02brq_WJ2.fits
NARG: FILENAME:           12  iffp02bsq_WJ2.fits
NARG: FILENAME:           13  iffp02btq_WJ2.fits
NARG: FILENAME:           14  iffp02buq_WJ2.fits
NARG: FILENAME:           15  iffp02bvq_WJ2.fits
NARG: FILENAME:           16  iffp02bwq_WJ2.fits
```

Step 2

Prepare the IN.* files taken in by the .src scripts in 01.XYM and 02.CMD directories. This step is important since the IN.* files in each directory are used to provide input to the backend Fortran scripts.

In [5]:
```
reduce.data_prep(directory)
```

Creature MATCHUP files

In the cell below we will achieve the following outcomes:

We will use respective PSF files for each filter to create a .XYM file for each exposure. A library PSF model is used to produce ".XYM" files, containing the star positions and magnitudes for each HST image exposure.
Generate output files TRANS.xym2mat, along with 16 MAT.0 files. Note that the first line of the IN.xym2mat and files beginning with 00 just defines the reference coordinates to be used for the output, so the 00 file does not need to be a *_WJ2.xym file.
Find all the objects that could be found in at least 12 out of 16 exposures.

In [6]:
```
reduce.MATCHUP_files(directory)
```

Let's open the MATCHUP file to inspect it further. The MATCHUP file is stored in 01.XYM/F814W for the F814W filter and 01.XYM/F606W for the F606W filter

THe MATCHUP file we will open will be for the F814W folder. You can uncomment the filename_606W line in the cell below to look at the F606W MATCHUP file.

In [28]:
```
filename_814W = Path(directory).resolve()/f"01.XYM/F814W/MATCHUP.F814W.XYM.02"
filename_606W = Path(directory).resolve()/f"01.XYM/F814W/MATCHUP.F814W.XYM.02"

cols = ["xbar", "ybar", "mbar", "xsig", "ysig", "msig", "qbar", "Nf", "Ng", "Nm", "Nmin", "Nstar", "pki", '
df = pd.read_csv(filename_814W, sep=r"\s+", comment="#", header=None, names=cols)
```

In [29]:
```
df
```

Out[29]:

	xbar	ybar	mbar	xsig	ysig	msig	qbar	Nf	Ng	Nm	Nmin	Nstar	pki	pkj	pkp	pk
0	28.0816	81.1979	-9.1099	0.0246	0.0243	0.0695	9.999	10	8	9	8	N000001	2527	2580	8	
1	41.7676	93.0470	-9.2503	0.0141	0.0173	0.0416	9.999	10	9	10	8	N000002	2541	2592	10	1
2	226.1286	94.6584	-9.3269	0.0121	0.0089	0.0246	9.999	10	9	10	8	N000003	2725	2594	10	
3	149.2051	97.4896	-7.7537	0.0338	0.0491	0.0525	9.999	9	9	8	8	N000004	2648	2597	10	1
4	317.4594	98.0957	-8.4755	0.0250	0.0340	0.0233	9.999	10	9	10	8	N000005	2816	2597	8	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
2254	726.6342	1127.6737	-3.8313	0.0793	0.0669	4.3817	9.999	8	8	8	8	N002255	3226	3627	10	
2255	945.2009	1131.1170	-9.1359	0.0325	0.0300	0.0314	9.999	10	10	9	8	N002256	3444	3630	10	1
2256	921.8165	1133.3954	-5.6098	0.0935	0.0667	3.2060	9.999	8	8	8	8	N002257	3421	3632	10	
2257	873.2732	1135.0444	-4.8129	0.0289	0.0247	5.5017	9.999	8	8	8	8	N002258	3372	3634	10	
2258	960.0142	1137.3410	-8.5449	0.0369	0.0356	0.0197	9.999	8	8	8	8	N002259	3459	3636	10	

2259 rows × 17 columns

From the MATCHUP file above, here are the most important things to note:

1. xbar and ybar correspond to the x and y star positions (in pixel coordinates) for stars in the HST images.
2. mbar is the F814W I magnitude of the stars.
3. xsig, ysig, msig are the errors on positions and magnitudes.

Using the MATCHUP files, we are ready to create a stack of the HST exposures.

Step 3

Create a file called "outpuq.fits". This fits file is a stack of your HST exposures. You can use the output fit files to identify your target. This step will prepare the output stack in both filters (F814W and F606W)

In [21]:
```
reduce.run_output_stack(directory)
```

```
WARNING: VerifyWarning: Found a SIMPLE card but its format doesn't respect the FITS Standard [astropy.io.fi
ts.hdu.hdulist]
WARNING: VerifyWarning: Found a SIMPLE card but its format doesn't respect the FITS Standard [astropy.io.fi
ts.hdu.hdulist]
WARNING: VerifyWarning: Found a SIMPLE card but its format doesn't respect the FITS Standard [astropy.io.fi
ts.hdu.hdulist]
WARNING: VerifyWarning: Found a SIMPLE card but its format doesn't respect the FITS Standard [astropy.io.fi
ts.hdu.hdulist]
WARNING: VerifyWarning: Found a SIMPLE card but its format doesn't respect the FITS Standard [astropy.io.fi
ts.hdu.hdulist]
WARNING: VerifyWarning: Found a SIMPLE card but its format doesn't respect the FITS Standard [astropy.io.fi
ts.hdu.hdulist]
WARNING: VerifyWarning: Found a SIMPLE card but its format doesn't respect the FITS Standard [astropy.io.fi
ts.hdu.hdulist]
WARNING: VerifyWarning: Found a SIMPLE card but its format doesn't respect the FITS Standard [astropy.io.fi
ts.hdu.hdulist]
WARNING: VerifyWarning: Found a SIMPLE card but its format doesn't respect the FITS Standard [astropy.io.fi
ts.hdu.hdulist]
```

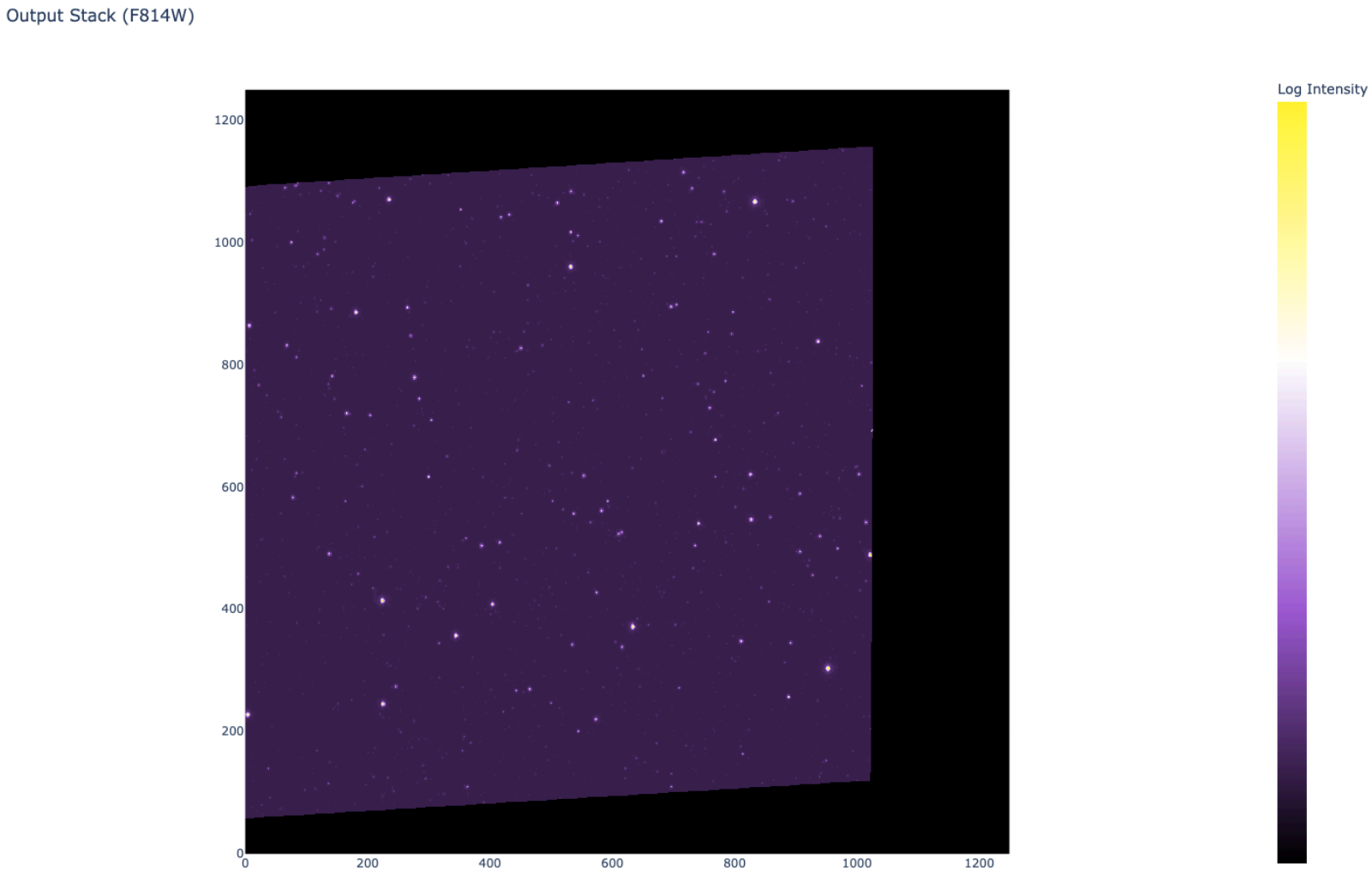
You can view the output stack in the jupyter notebook here. The output stack for the F814W filter is kept in 01.XYM/F814W and the output stack for the F606W filter is kept in 01.XYM/F606W.

We open the output stack for F814W below. You can uncomment the filename_606W line in the cell below to look at the F606W output stack.

The scaling can be adjusted depending on your preference by changing the argument given to "ImageNormalize" in the cell below.

In [35]:
```
filename_814W = Path(directory).resolve()/f"01.XYM/F814W/outputq_F814W.fits"
#filename_606W = Path(directory).resolve()/f"01.XYM/F814W/outputq_F606W.fits"

hdul = fits.open(filename_814W)
data = hdul[-1].data
hdul.close()
data = np.nan_to_num(data, nan=0.0, posinf=0.0, neginf=0.0)
```

```
norm = ImageNormalize(data, stretch=AsinhStretch(0.02))
scaled = norm(data)

nonbinary_colors = custom_cmap.nb_colors()

fig = px.imshow(scaled, origin='lower', color_continuous_scale=nonbinary_colors, title="Output Stack (F814W

fig.update_layout(width=1000, height=1000, coloraxis_colorbar=dict(title="Log Intensity", tickvals=[]))
fig.show()
```


Output Stack (F814W)
Log Intensity
1200
1000
800
600
400
200
0
0
200
400
600
800
1000
1200

In [ ]: